

Competitive Programming

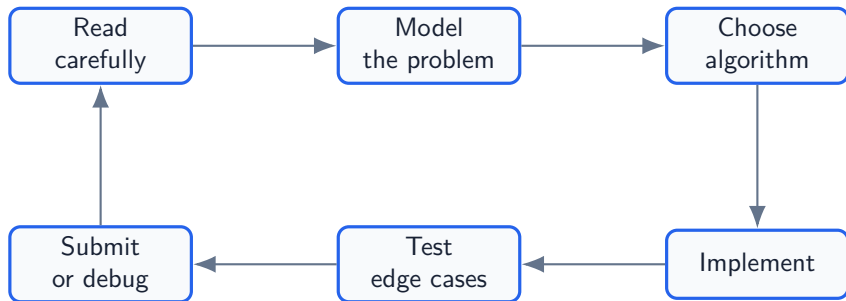
Bootcamp Day 1

Algorithms, Data Structures, and Problem Solving

Ryan Pelchat

August 2026

1. Problem-solving workflow
2. Key data structure or algorithm
3. Worked example
4. Implementation patterns
5. Practice problems



Problem: Replace this with the problem name.

Input

Array of n integers

Output

One answer

Constraints

$1 \leq n \leq 2 \cdot 10^5$

Target

$\mathcal{O}(n \log n)$ or better

Key Observation

State the idea that makes the problem easier. For example: sort first, use prefix sums, turn it into a graph, or maintain an invariant.

1. Preprocess the input.
2. Maintain the right state while scanning.
3. Update the answer greedily / dynamically / using a data structure.
4. Prove the invariant and analyze complexity.

Time: $\mathcal{O}(n \log n)$ **Memory:** $\mathcal{O}(n)$

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n;
11    cin >> n;
12
13    vector<int> a(n);
14    for (int &x : a) cin >> x;
15
16    // TODO: solve the problem
17
18    cout << 0 << '\n';
19    return 0;
20 }
```

```
1 import sys
2
3 def solve() -> None:
4     input = sys.stdin.readline
5     n = int(input())
6     a = list(map(int, input().split()))
7
8     # TODO: solve the problem
9
10    print(0)
11
12 if __name__ == "__main__":
13     solve()
```

Invariant

After processing the first i elements, our data structure contains exactly the information needed to answer queries involving those elements.

Why it works

Use induction: show that the invariant is true initially, remains true after each update, and implies the final answer.

Topic	Easy	Medium
Prefix sums	Static Range Sum	Haybales
Greedy	Interval Scheduling	Dragon of Loowater
Graphs	BFS Grid	Shortest Path
DSU	Union Find	Virtual Friends
DP	Fibonacci	Knapsack

- ▶ Identify constraints before choosing an algorithm.
- ▶ Look for transformations: sorting, graphs, prefix sums, DP states.
- ▶ Write down the invariant before coding.
- ▶ Test tiny cases, edge cases, and worst cases.

Questions?

Let's solve problems.